

Projekt do předmětu KNN
Zpracování obsahů z naskenovaných dokumentů

Finální odevzdání

Jakub Vlk (xvlkja07)
Veronika Nevařilová (xnevar00)
Emma Krompaščíková (xkromp00)

1 Odkaz na repozitář

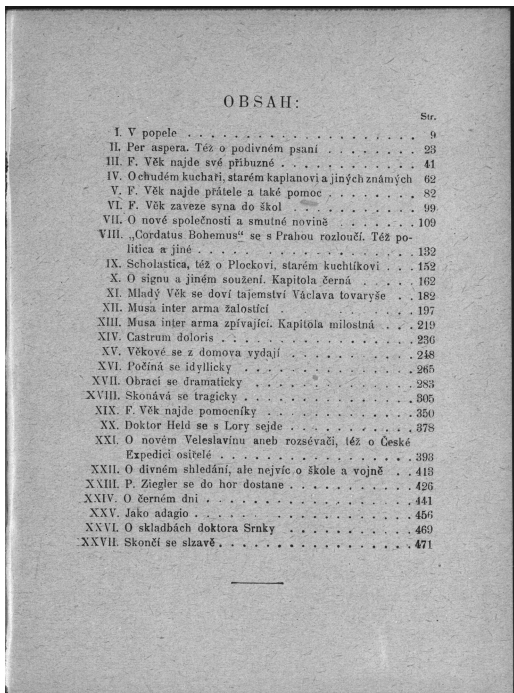
Repozitář je k dispozici na této adrese: <https://github.com/vlccek/knn-project>. V repozitáři je soubor `readme.md`, který popisuje, co se kde nachází.

2 Definice úlohy

Cílem tohoto zadání je vytvořit systém pro tvorbu databázových záznamů z naskenovaných stránek obsahů pro jejich jednodušší zpracování - tedy extrahovat z obrázků informace o jednotlivých kapitolách:

- Název kapitoly
- Číslo kapitoly (pokud je k dispozici)
- Číslo strany
- Vztah kapitoly k ostatním kapitolám – její podkapitoly (*children*, viz příklad požadovaného výstupu), pokud nějaké má

Vstupem je tedy sken stránky knihy (přímo obsah, popřípadě strana, na které se nachází název nějaké kapitoly). Výstupem je soubor popisující kapitoly dané stránky ve formátu JSON.



Obrázek 1: Příklad stránky obsahující pouze obsah.



Obrázek 2: Příklad stránky obsahující nadpisy s textem.

Požadovaný výstup pro stránku z obrázku 1 by pak vypadal následovně:

```
"result": [{
  {
    "title": "V popele",
    "chapter_number": "I",
    "page_number": "9",
    "children": []
  },
  {
    "title": "Per aspera. Též o podivném psaní",
    "chapter_number": "II",
    "page_number": "23",
```

```
"children": []
}...
]
```

3 Existující řešení

Při analýze existujících řešení jsme na žádné, které by řešilo přesně naše téma, nenatrefili. Proto jsme v této sekci sepsali řešení, která jsou našemu problému nejbližší.

3.1 DONUT

DONUT [1], z angl. *Document Understanding Transformer*, je end-to-end model pro porozumění dokumentů, který pro jejich zpracování nepoužívá OCR. Přímo od autorů jsou k dispozici modely DONUT přetrénované na specifické typy úloh, které zahrnují:

- **Zpracování dokumentů** – výstupem modelu je struktura a informace extrahované z dokumentu z obrázku v digitální podobě ve formátu JSON. Tento model byl trénován na podmnožině datasetu CORD¹, který obsahuje účtenky, převážně z restaurací. Dataset sestává z 800 záznamů pro train, 100 záznamů pro validation a 100 záznamů pro test množinu. Úspěšnost modelu DONUT na tomto datasetu (metrika *Tree Edit Distance*) je podle oficiálního GitHub repozitáře² 91,3 %.
- **Klasifikace dokumentů** – určování typu dokumentu – např. `article`, `receipt` aj.
- **Zodpovídání otázek týkajících se dokumentů** – např. kolik položek na dané účtence je, nebo celková částka k zaplacení na dané faktuře.

Tento model byl také prezentován ve videu, konkrétně původní článek *OCR-free Document Understanding Transformer* [1].

3.2 PERO OCR + LayoutLMv3

Jedná se o řešení, které vyžaduje kombinaci dvou odlišných projektů:

- **PERO OCR** – nástroj, pomocí kterého lze automaticky přepisovat tištěný a ručně psaný text do digitální podoby spolu s dalšími informacemi [2]. Jedna z velkých výhod oproti modelu DONUT je zaměření na český jazyk.
- **LayoutLMv3** – model zpracování dokumentů pro různé účely [3]. Na jeho vstup by mělo být možné vložit výstup z PERO (text, boxy ohraničující text) a spolu s původním obrázkem dotrénovat model pro požadovaný výstup. Nevýhodou však je maximální vstupní sekvence 512 tokenů, což by pro hodně obrázků z datasetu představovalo problém.

4 Dataset

Pro trénování byla obdržena od vedoucího projektu sada obrázků se skeny obsahů knih a jiných papírových médií a anotací k nim, která není veřejně dostupná. Konkrétní datasety s požadovanými výstupy jsme si však museli vytvořit sami. Naše výsledné datasety jsou k dispozici na [tomto](#) odkazu ve formátu tarballu.

¹<https://huggingface.co/datasets/naver-clova-ix/cord-v2>

²<https://github.com/clovaai/donut>

HLAVA III.	Str.
§ 19. Uzavření obchodů	68
HLAVA IV.	
§ 20. Plnění obchodů	74
B. ČÁST ZVLÁŠTNÍ.	
HLAVA I.	
O tržové smlouvě obchodní (čl. 337—359 obch. zák.)	
§ 21. Pojem a uzavření tržové smlouvy obchodní. Koupě na zkoušku	78
§ 22. O povinnostech prodávatele ze smlouvy tržové	81
§ 23. O povinnostech kupitele ze smlouvy tržové	84
§ 24. O správě při obchodní smlouvě tržové	86
§ 25.—27. O prodlení kontrahentů při obchodní smlouvě tržové	92
§ 28. O vlivu konkursu na obchodní smlouvu tržovou	101
§ 29. Dodatek o jednáních bursovních	102
HLAVA II.	
O kupecké komisi (čl. 360—378 obch. zák.)	
§ 30. Pojem a předmět kupecké komise	105
§ 31. Povinnosti komisionáře ze smlouvy komisionářské	106
§ 32. Povinnosti komitentovy ze smlouvy komisionářské	110
§ 33. Komisionář jako samokontrahecent	112
HLAVA III.	
§ 34. O spedici	115
HLAVA IV.	
O smlouvě nákladní (dovozní).	
Oddíl 1.	
O smlouvě nákladní (dovozní) vůbec.	
§ 35. 1. Pojem smlouvy nákladní a její uzavření. Listiny při ní obvyklé	121
§ 36. 2. Právní povinnosti dovozníky ze smlouvy nákladní	124
§ 37. 3. Práva dovozníky ze smlouvy nákladní	131
§ 38. 4. O právním postavení příjemce	134
§ 39. 5. O náložném číli nákladním listu (Ladeschein) zvlášť	136
Oddíl 2.	
Hlavní zvláštnosti ve právní úpravě železničního poměru nákladního.	
§ 40. 1. Právní normy upravující železniční smlouvu nákladní	138
§ 41. 2. Uzavření železniční smlouvy nákladní. Tarif. Listiny smlouvy	142
§ 42. 3. Některé zvláštnosti co do práv a závazků ze železniční nákladní smlouvy	145

Obrázek 3: Ukázka nejkomplexnějšího vzorku v rámci datasetu z hlediska množství zanoření.

4.1 Poskytnuté anotace

Anotace spočívají v označení oblastí, ve kterých se nachází jednotlivé komponenty k extrakci. Tyto anotace byly využity pro variantu PERO OCR + LayoutLMv3. Jmenovitě dataset obsahuje:

- Název kapitoly (a jestli se jedná o úroveň prvního nebo jiného stupně)
- Číslo kapitoly (pokud je k dispozici)
- Číslo strany
- Relace mezi kapitolou a číslem stránky

VII.	Příprava k hotovení řezů jednotlivých a seriových	76
	Metoda zalévání preparátů do celloidinu	76
	Metoda zalévání preparátů do paraffinu	79
	Zalítí preparátů do celloidin-paraffinu	84

Obrázek 4: Ukázka vizualizace anotací. Modře jsou znázorněny nadpisy hlavní úrovně, zeleně nadpisy nižší úrovně, žlutě čísla kapitol a červeně čísla stránek. Relace jsou označeny šipkami.

Velikosti obrázků	
Minimum	500x700 px
Maximum	6645x9447 px
Medián	1500x2409 px

Tabulka 1: Informace o velikosti vstupních obrázků.

4.2 Dataset pro DONUT

Pro účely trénování DONUT modelu je potřeba mít připravené dvojice obrázek-očekávaný JSON. Původně jsme chtěli vytvořit JSON pomocí poskytnutých anotací. Zamýšleli jsme obrázek rozřezat

podle poskytnutých anotací, na jednotlivé výřezy aplikovat OCR a tyto výsledky vložit do formátu JSONu definovaného výše. Problémem tohoto přístupu bylo, že anotace poskytují pouze relace mezi nadpisy a čísly stránek. Chyběly relace mezi úrovněmi nadpisů a také relace mezi číslem kapitoly a nadpisem.

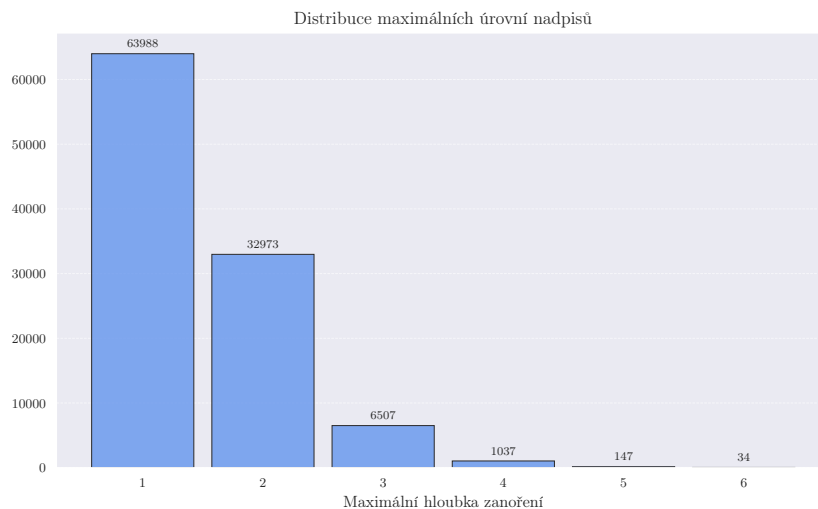
Tento problém by šlo vyřešit pomocí přístupu založeného na nejbližším sousedovi, ale rozhodli jsme se pro použití API velkého jazykového modelu (LLM), konkrétně Gemini³. Tento přístup dával tak dobré výsledky, že jsme pro později zvolený baseline model, DONUT, nepoužívali poskytnuté anotace vůbec.

Kategorie	Detaily
Velikost datasetu	
Train	4 258 vzorků
Test	541 vzorků
Validation	535 vzorků
Celkem	5 334 vzorků
Jazyky (Seřazeno dle četnosti)	Čeština, Staročestina, Němčina, Latina, Angličtina, Italština, Slovenština
Maximální rekurze	
Maximum napříč datsety	6
Maximální počet tokenů	6 556

Tabulka 2: Přehledová tabulka se základními informacemi o datasetu pro DONUT.

V tabulce 4 jsou uvedeny základní informace o datasetu vygenerovaném pomocí LLM. Dataset obsahuje více než 5 000 vzorků rozdělených do trénovací, testovací a validační množiny, přičemž pokrývá sedm různých jazyků. Distribuci klíčových charakteristik datasetu, konkrétně hloubku rekurze a počet tokenů, lze vidět na obrázcích 5 a 6. Počet tokenů je důležitý, protože model DONUT je model založený na architektuře Transformers a používá se tokenizace⁴.

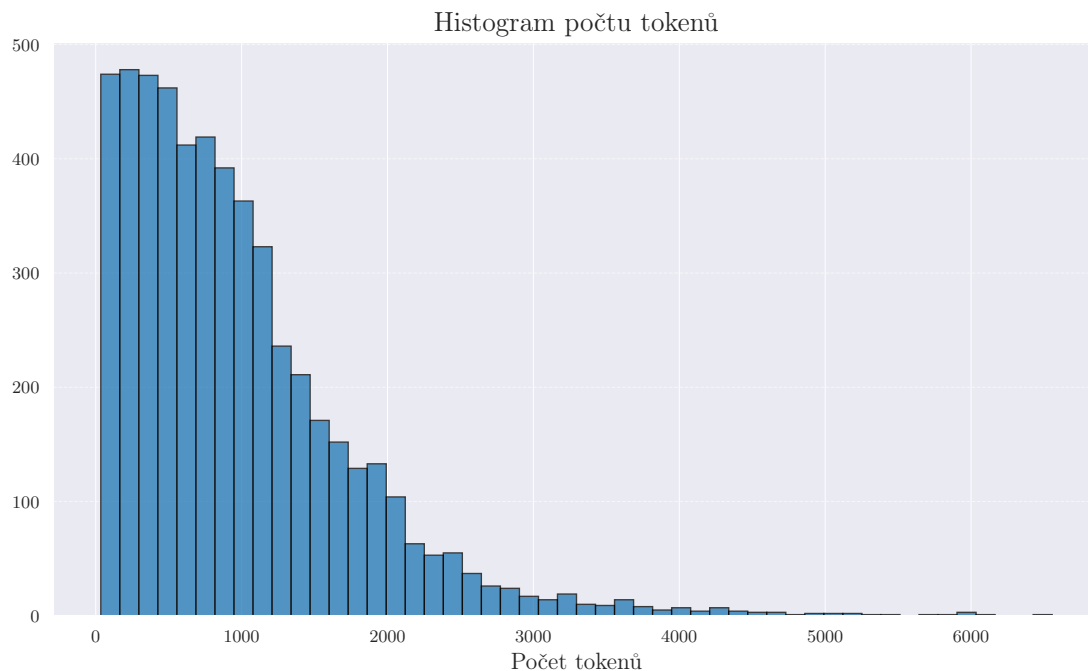
Tento přístup i spolehlivě identifikuje stránky, na kterých nejsou obsahy. LLM vrací pro takové stránky prázdný seznam, podle promptu, který jsme zadali. Tyto stránky jsme dále pro trénování nijak nevyužili.



Obrázek 5: Histogram pro maximální úroveň nadpisů.

³Používali jsme model GEMINI-2.0-FLASH, prompt je možné najít v Github repozitáři.

⁴Používali jsme tokenizátor pro DONUT



Obrázek 6: Histogram s počtem tokenů po tokenizaci.

4.3 Dataset pro LayoutLMv3

Pro LayoutLMv3 jsou na vstupu potřeba jednotlivá slova, jejich bounding boxy a samotný obrázek. Očekávaným výstupem modelu je seznam klasifikací jednotlivých slov ve stejném pořadí, v jakém byla na vstupu. Pro trénování pak LayoutLMv3 vyžaduje na vstupu navíc ještě právě tyto anotace správných klasifikací. Obecně je pro trénování potřeba dataset ve formátu:

```
[
  {
    "tokens": ["V", "popelé", ...],
    "bboxes": [[392, 367, 549, 392], [142, 418, 165, 440], ...],
    "ner_tags": [1, 0, ...],
    "image": "path/to/image.jpg"
  } ...
]
```

V `ner_tags` jsme použili mapování jednotlivých tříd stejných jako v dodaných ručních anotacích na celá čísla:

```
"trash": 0
"kapitola": 1
"cislo strany": 2
"jiny nadpis": 3
"jine cislo": 4
"podnadpis": 5
"nadpis v textu": 6
```

Kromě tříd vyskytujících se v ručních anotacích jsme pak přidali ještě speciální třídu `trash` s hodnotou 0, která vyjadřuje slova, která jsou pro vytváření obsahu nedůležitá (dodatky na konci strany či jiný text nepředstavující část obsahu).

Na doporučení vedoucího jsme použili nástroj PERO⁵, jehož výstupem jsou mimo jiné také soubory ve formátu ALTO (Analyzed Layout and Text Object). Vizualizaci takového souboru lze vidět na obrázku 7. Jedná se o standardní XML formát pro popis rozložení a obsahu digitalizovaných dokumentů, včetně umístění textových bloků, řádků a slov, a rozpoznaného textu. Z tohoto souboru lze získat požadované informace, konkrétně jednotlivá slova a jejich bounding boxy (BB).

⁵<https://github.com/DCGM/pero-ocr>

Obsah.		Strana
Úvod		VII
Opravy a doplnky k úradným názvom obcí, osád a miestnych častí obcí (ku str. 3—155)		XV
Tab. I. Obyvateľstvo správnych okresov Slovenska podľa národnosti resp. materinskej reči v rokoch 1880—1921 (v číslach pomerných)		XVI
Tab. II. Obyvateľstvo správnych okresov Slovenska podľa náboženského vyznania v rokoch 1900—1921 (v číslach pomerných)		XVIII
Zkratky a vysvetlenie značiek		XXI
Hlavná časť: Obce, osady, miestne časti obcí a časti miest		3
Obce ležiace na západ od demarkačnej línie, avšak spravované civilnou správou Podkarpatskej Rusi		156
Obce ležiace na východ od demarkačnej línie, avšak spravované župným úradom v Košiciach		156
Dodatky:		
I. Prehľad Slovenska podľa žúp		159
II. Prehľad Slovenska podľa správnych okresov		159
III. Prehľad Slovenska podľa súdnych okresov		161
IV. Vojenské posádky a ústavy		164
V. Počet notariátov, veľkých a malých obcí, osád, v tom mest		167
I. Zmeny v správnom rozdelení Slovenska od 15. februára 1921 do 15. januára 1927, na ktoré bol v texte už vzatý zreteľ		171
II. Zmeny v správnom rozdelení Slovenska nastalé v čase tlače, na ktoré nebol v texte vzatý zreteľ		176
III. Zmeny, obce, v ktorých bolo 15. februára 1921 vykonané čl. sčítanie ľudu, avšak neskôr boli posúpené susedným štátom		176
Abecedný soznam miest na Slovensku		179
Abecedný soznam súdnych okresov na Slovensku		277
Tlačové chyby		278

Obrázek 7: Vizualizace ALTO formátu. Lze vidět ohraničení pro části textu, řádky a slova.

Celý proces vytváření datasetu tedy probíhal takto:

1. Zpracování obrázků pomocí PERO
2. Převod černobílých obrázků na RGB formát (LayoutLMv3 vyžaduje RGB obrázky)
3. Zpracování ALTO výstupů a převod souřadnic na souřadnice kompatibilní s těmi z ručních anotací
4. Porovnání bounding boxů jednotlivých slov s ručními anotacemi - daný bounding box byl klasifikován stejnou třídou, jakou má bounding box z ručních anotací, se kterou se nejvíce překrýval. Pokud se nepřekrýval s žádným boxem z anotací, znamená to, že dané slovo není částí obsahu a je tedy klasifikováno jako třída **trash**.
5. Normalizace souřadnic bounding boxu - LayoutLMv3 akceptuje souřadnice v rozmezí hodnot 0-1000.

Vzhledem k tomu, že u tohoto přístupu jsme na rozdíl u Donutu byli omezeni počtem obrázků, který byl anotován, po vyřazení některých nevalidních vzorků (obrázky s obsahem zanořeným více než 2 úrovně, anotace s neodpovídajícími rozměry obrázku vůči rozměrům uvedeným ve výstupech z PERO) má výsledný dataset pouze okolo 1000 obrázků se zvoleným poměrem množin 80:10:10:

Množina datasetu	Počet vzorků
Train	784
Test	98
Validation	98
Celkem	980

Tabulka 3: Velikost jednotlivých množin datasetu.

5 Použité metriky

Převzali jsme metriky, které používali autoři původního modelu DONUT pro evaluaci modelu pro zpracování úctenek, což je typ úkolu nejvíce podobný našemu. Jedná se o metriky *Tree Edit Distance* a *F1*.

5.1 Normalizovaná Tree Edit Distance přesnost

Pro vyhodnocení strukturální podobnosti mezi predikcí modelu a očekávaným výstupem jsme využili normalizovaný Tree Edit Distance (TED). TED sama o sobě počítá minimální počet operací (vlození, smazání a substituce uzlů), které jsou potřeba k transformaci jednoho stromu na druhý. V našem případě jsou při testování DONUT modelu převedeny vygenerované JSON struktury na stromy, kde každý uzel reprezentuje jednu položku obsahu. Nižší hodnota TED značí větší podobnost mezi predikcí a referenčním výstupem, což naznačuje lepší schopnost modelu zachytit hierarchickou strukturu obsahu dokumentu. Normalizace podle velikosti referenčního stromu a odčítání této hodnoty od 1 pak představuje přesnost modelu pohybující se v hodnotách od 0 do 1. Tuto konkrétní metriku proto maximalizujeme.

$$\text{NormedTEDAcc} = \max \left(0, 1 - \frac{\text{TED}(\text{prediction}, \text{answer})}{\text{TED}(\emptyset, \text{answer})} \right)$$

5.2 F1 skóre

Jako doplňkovou metriku jsme použili F1 skóre, které poskytuje vyvážený pohled na přesnost a úplnost extrakce. F1 skóre je harmonický průměr mezi precision (přesnost) a recall (úplnost), kde precision představuje podíl správně identifikovaných položek v predikci a recall představuje podíl referenčních položek, které byly úspěšně predikovány. Tato metrika je důležitá pro vyhodnocení schopnosti modelu korektně extrahovat všechny relevantní komponenty obsahu (nápisy, čísla stránek, čísla kapitol) bez falešných detekcí. Vyšší hodnota F1 skóre indikuje kvalitu extrakce obsahů a tuto metriku proto rovněž maximalizujeme. Hodnoty taktéž náležejí do intervalu od 0 do 1.

$$F1 = \frac{2 \cdot \text{precision} \cdot \text{recall}}{\text{precision} + \text{recall}}$$

Je také potřeba zmínit, že evaluace modelu DONUT v kódu probíhá tak, že se vyhodnocuje hodnota F1 skóre na výsledcích predikce jednotlivých obrázků. Nejmenší jednotka tohoto vyhodnocení je tedy celá predikce jednoho obrázku. Pokud je predikce naprosto stejná s požadovaným výsledkem, je to klasifikováno jako *true positive*. Pokud se v predikci liší byť jen jeden znak, je predikce označena za *false positive* nebo *false negative*, v závislosti na situaci.

6 Baseline řešení

Jako baseline řešení jsme z možností zmíněných v sekci 3 zvolili model DONUT z důvodů:

- **End-to-end řešení** – není potřeba spojovat více modelů za sebe
- **Trénovaný na velmi podobný typ úkolu** – přetrénovat CORD verzi na náš požadovaný výstup by mělo být možné, a díky tomu, že model byl přetrénován na podobném úkolu, nebude pravděpodobně potřeba mít obrovský dataset – model již bude umět zpracovávat obrázek, co se zanoření a určování prvků v obrázku týče
- **Úspěšnost v přetrénování modelu na podobných úkolech** – při hledání informací o modelu jsme natrefili na spoustu internetových diskuzí, kde použití CORD DONUT modelu pro specifická řešení vývojáři chválili
- **Existující trénovací a testovací skripty** – autoři ve svém GitHub repozitáři poskytují skripty pro práci s modelem

DONUT má řadu nevýhod, které mohou trénování ztížit nebo přímo znemožnit, mezi které patří zejména to, že **nebyl trénován na českém jazyce**. DONUT byl trénován pouze na angličtině, japonštině, čínštině a korejštině. Zkusili jsme proto originálnímu modelu verze CORD předložit některé obrázky z našeho datasetu a ten nepřekvapivě přepisoval všechny znaky obsahující interpunkci bez ní. Každopádně v tokenizeru modelu jsme našli pár českých znaků, a proto jsme se i přesto trénování rozhodli vyzkoušet.

6.1 Dataset pro DONUT

Architekturu datasetu jsme doladili podle požadavků GitHub repozitáře DONUT modelu. Dataset se skládá ze tří adresářů, **train**, **validation** a **test**, přičemž každý adresář obsahuje soubor `metadata.jsonl`, ve kterém se nachází dvojice `file_name` a `ground_truth` pro každý obrázek z daného datasetu. Ve stejném adresáři se pak nachází také všechny obrázky zmíněné v metadatach. Jako `ground_truth` jsme použili přímo požadované JSON výstupy modelu ve formě textového řetězce.

6.2 Trénování

Pro trénování jsme si vytvořili účty na Metacentru⁶. DONUT totiž potřebuje pro trénování velký objem GPU paměti, a proto lokální trénování nebylo možné.

I přesto, že samotné trénování mělo být poměrně jednoduché, jsme natrefili na spoustu problémů, které nakonec vyústily ve velké změny celého kódu DONUT repozitáře. Trénování jsme úspěšně zprovoznili, ale i přesto, že model v průběhu trénování vykazoval známky učení a zlepšování se, měl po natrénování vždy mnohem horší úspěšnost než měl jeho koncový stav trénování. Tento problém je (dle množství vytvořených issues s tímto tématem na DONUT GitHubu) poměrně častý a neexistuje univerzální postup, kterým by se vyřešil. Museli jsme proto upravovat kód, instalovat různé verze balíčků a zkoušet různá nastavení trénování. Nakonec se však kód podařilo dostat do bodu, kdy trénování začalo mít mnohem lepší výsledky.

Před našimi úpravami trvalo i 15 epoch modelu, aby začal rozpoznávat strukturu požadovaného výstupu v obrázcích. V aktuální verzi již po jedné epoše vykazuje orientaci v tom, jakou úlohu má plnit a správně dokáže například spojit název kapitoly a číslo její strany.

Další věcí, která nás mile překvapila, bylo naučení správného rozpoznávání českých znaků a textů, s čímž model po pár epochách neměl žádný větší problém.

Ze začátku jsme zkoušeli model trénovat na 8 epochách, a po nich již model bez problému dokáže některé obrázky zpracovat se 100% úspěšností.

6.3 Vyhodnocení baseline

Vyhodnocovat původní DONUT model CORD na našich datech nemělo smysl. Daný model je totiž naučen na účtenkách, a proto i pokud našel v našich datech nějaký vzor, výstupní JSON absolutně neodpovídal našemu požadovanému výstupu a obě metriky by očividně měly hodnotu 1. Proto jsme model vyhodnotili po přetrénování, abychom měli relevantní výsledky.

Vyhodnocení jsme provedli standardní inferencí na modelu. Použili jsme testovací dataset. Výsledné hodnoty TED a F1 po natrénování byly:

Počet epoch	Norm. tree edit distance přesnost	F1 skóre
8	0.70	0.39

Tabulka 4: Tabulka s výsledky inference na modelu vztažených k počtu epoch.

Tyto prozatimní výsledky vnímáme pozitivně, vzhledem k tomu, že jsme čelili problémům s baseline řešením a zatím jsme tedy nemli možnost vyladit různé konfigurace modelu a vyzkoušet více epoch. Pouhých 8 epoch stačilo modelu, který nebyl předtím trénován na českém jazyce, aby začal přepisovat naše skeny obrázků s přesností norm. TED 70 % a F1 39 %. ⁷

6.4 Vývoj loss

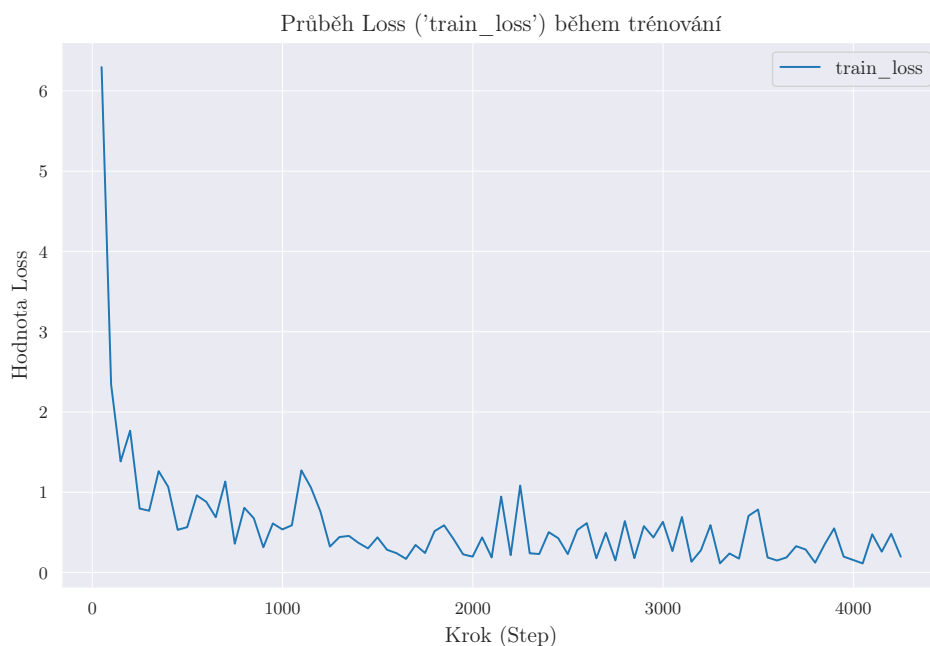
Z průběhu loss funkce na obrázku 8 jde vidět, že by určitě šlo model ještě trénovat a její hodnoty stále lehce klesají. Loss funkce vykazuje zdravé učení.

6.5 Úpravy baseline po checkpointu

Po odevzdání checkpointu jsme provedli ještě několik experimentů s cílem vylepšit přesnost modelu. Vyfiltrovali jsme dataset tak, aby obsahoval jen nadpisy druhé úrovně. Experimentovali jsme také s počtem epoch, který jsme tentokrát nastavili na 25. Výsledky se nám podařilo mírně vylepšit, jak je vidět v tabulce 5. Provedli jsme i ruční kontrolu výstupu jazykového modelu a kontrola ukázala,

⁶<https://metavo.metacentrum.cz/>

⁷Model je nahrán na Google Drive: https://drive.google.com/drive/folders/1wHiaRq4rd3egE_rcKunOHj6pBsMckQd?usp=sharing.



Obrázek 8: Vývoj loss funkce v průběhu trénování.

že většina kontrolovaných vzorků byla zcela bezchybná. U malé části vzorků, které vykazovaly určité nedostatky, jsme identifikovali několik hlavních kategorií chyb.

Přibližně třetinu těchto problémových vzorků tvořily chyby v interpunkci. Jednalo se především o chybějící tečky za římskými čísly, ale zaznamenali jsme i nesprávné použití či absenci pomlček a spojovníků, nebo nadbytečná interpunkční znaménka.

Podobně časté, rovněž s podílem přibližně jedné třetiny problémových vzorků, byly strukturální nesrovnalosti. Zde jsme narazili na extrakci nadbytečných dat, typicky v podobě nadbytečných čísel stran, která nepatřila k danému záznamu, nebo i celých řádků obsahu, které do extrahovaných informací nepatřily. Naopak, v některých případech chyběly relevantní části textu kapitol. Specifickým problémem byla také občasná mylná extrakce slova "OBSAH" jako názvu kapitoly. Jako méně častá, avšak docela závažná chyba, byly posunuté čísla stránky obsahu o jeden řádek nahoru, kdy došlo k chybnému strukturálnímu přiřazení celého obsahu stránky.

Menší podíl tvořily problémy související s optickým rozpoznáváním znaků a zpracováním textů. Zde jsme zaznamenali nesprávně rozpoznané znaky, zejména s diakritikou (např. SIRÉNA rozpoznáno jako SIRENA), záměnu písmen (např. D jako O), či potíže při zpracování cizojazyčných kaligrafických německých textů, kde bylo náročné zjistit, zda je okrasné písmo rozpoznáno správně. Problém představoval i ručně psaný text, který nebyl spolehlivě rozpoznán.

Nakonec, s podílem okolo 5–10 % problémových vzorků, se objevovaly chyby při spojování slov, která byla v obsahu rozdělena na konci řádku.

I přesto je však, vzhledem ke způsobu, jakým je počítána hodnota F1 v evaluačním skriptu modelu DONUT (popsána v sekci 5.2), tato hodnota poměrně vysoká a znamená poměrně přesné zpracování obsahů stran.

Počet epoch	Norm. tree edit distance přesnost	F1 skóre
25	0.75	0.42

Tabulka 5: Tabulka s výsledky inference na modelu vztažených k počtu epoch v druhé části semestru.

7 PERO-LayoutLMv3

Projekt byl pojat jako porovnání dvou odlišných přístupů k řešení daného problému. První z nich, založený na modernějších principech Transformerů, který byl prezentován v rámci checkpointu, vyžaduje náročnější anotace. Jejich ruční vytvoření by bylo velmi drahé a náchylné k chybám, a

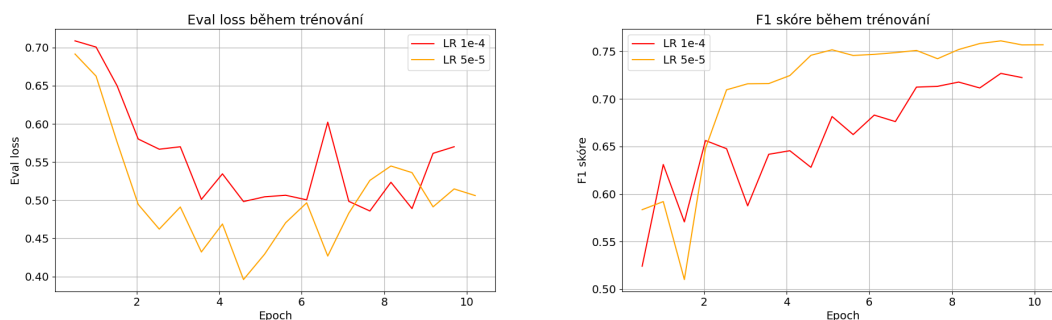
proto jsme se rozhodli je vygenerovat pomocí jazykového modelu. Tímto postupem se de facto destilují znalosti LLM, bohužel však s tím se přenáší i jeho chyby.

Druhý přístup sice rovněž vyžaduje anotace, ale jejich tvorba je jednodušší než vytvoření kompletního JSONu s úrovní a kompletním přepisem potřebným pro první přístup.

Celkově lze říci, že přístup založený na PERO a LayoutLMv3 klade menší nároky na přímou anotaci člověkem (případně se zbavuje závislosti na LLM), ale za to je potřeba vynaložit více úsilí na komplexnější práci se vstupy a výstupy modelu.

7.1 Trénování LayoutLMv3

Základ trénovacího skriptu byl převzat z GitHubu Hugging Face⁸, který byl dále upraven pro potřeby tohoto projektu. Vzhledem k velké vytíženosti Metacentra a faktu, že LayoutLMv3 nevyžaduje příliš velký výpočetní výkon, trénování tentokrát oproti Donutu probíhalo na Google Colab⁹ formou Jupyter notebooku. Trénování probíhalo několikrát s různým nastavením hyperparametrů. Nejvyšší přesnosti bylo však dosaženo při trénování 10 epoch a learning rate $5 \cdot 10^{-5}$. Při vyšším počtu epoch či větší learning rate již docházelo u modelu k přetrénování. Pro evaluaci při i po trénování bylo využito metriky F1. Trénováním bylo dosaženo maximální hodnoty F1 na testovací sadě 75,7 %. Vývoj hodnot loss funkce a F1 skóre je zobrazen v obrázku 9, kde jsou v grafu vyneseny pro srovnání dva vybrané běhy trénování s různým nastavením learning rate, $5 \cdot 10^{-5}$ (oranžově, výsledný zvolený model) a $1 \cdot 10^{-4}$ (červeně).



Obrázek 9: Vývoj hodnot loss funkce a F1 skóre na evaluační sadě během trénování s různým nastavením hodnot learning rate.

Po úspěšném natrénování LayoutLMv3 bylo již možné přejít ke spojení všech nástrojů ve formě výsledného kódu pro zpracování jednotlivých obrázků.

7.2 Pipeline pro inferenci

Výsledný kód pro zpracování by měl být co nejvíce uživatelsky přívětivý a jednoduchý. Vstupem by tedy ideálně měl být pouze daný obrázek, který uživatel chce převést na obsah, a výstupem by měl být již samotný obsah ve formátu JSON.

7.2.1 Spuštění

Soubor `run.sh` pro spuštění tohoto výsledného kódu je umístěn v `pero_layout/pipeline`. Obrázek, jehož obsah chce uživatel převést na JSON, musí být umístěn ve složce `images`. Dále se ve složce musí nacházet složky obsahující PERO a LayoutLMv3. Skript se spouští pouhým příkazem `./run.sh` (podrobný návod pro spuštění je přiložen v `readme.md`). Celé zpracování daného obrázku probíhá ve třech základních krocích popsaných níže.

7.2.2 Získání OCR výstupů pomocí PERO

V první části skript zavolá PERO pro zpracování daného obrázku v adresáři `images`. Všechny výstupy z PERO jsou uloženy v adresáři `output`. Pro další zpracování však jsou použity pouze výstupy ALTO.

⁸https://github.com/huggingface/transformers-research-projects/blob/main/layoutlmv3/run_funsd_cord.py

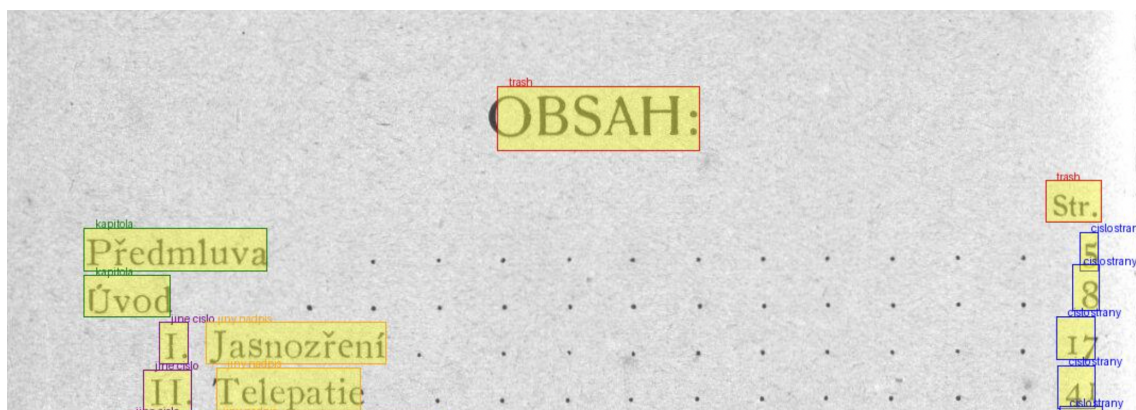
⁹<https://colab.research.google.com/>

7.2.3 Převod OCR výstupů do formátu kompatibilního s LayoutLMv3

Dále se spustí soubor `ocr-to-json.py` nacházející se v adresáři `processing`, kterému jsou předány dva argumenty – adresář s výstupy PERO a adresář obsahující původní obrázek. Tento skript spustí převod výstupů z PERO do formátu, který přijímá LayoutLMv3, pomocí třídy `PeroProcessor`. Ta funguje podobně jako skript pro přípravu datasetů – převede výstup OCR na pole jednotlivých slov, jim odpovídající normalizované bounding boxy, a předá také cestu k danému původnímu obrázku. Všechna potřebná data jsou tedy vrácena jako slovník s klíči `bboxes`, `tokens`, `image_path`.

7.2.4 Inference pomocí LayoutLMv3

Data popisující daný obrázek jsou předána třídě `LayoutProcessor`, která provede načtení natrénovaného modelu a inferenci obrázku. Následně je ještě provedeno zpracování, kdy se např. odstraní okrajové výplně výstupů či se spojí podslova jednotlivých slov. Pro kontrolu správnosti je ještě přidán kód, který jednotlivá klasifikovaná slova v obrázku označí a připíše k nim jejich klasifikovanou třídu. Tento bod slouží především k ověření, že celé zpracování probíhá korektně. Příklad takto zpracovaného obrázku je zobrazen v obrázku 10. Výstupem tohoto kroku je tedy pole klasifikovaných slov ve stejném pořadí, jako byly modelu předány, a níže zobrazený obrázek.



Obrázek 10: Vizualizace výřezu výstupu z LayoutLMv3 včetně vyznačených klasifikací. Tento bod slouží k ověření, že práce se souřadnicemi pomocí normalizace/denormalizace probíhala korektně a že jednotlivá slova jsou klasifikována správně.

V tomto kroku se ale také projevuje hlavní nevýhoda LayoutLMv3, a to ta, že vstup je omezen na pouhých 512 tokenů. Tento problém byl částečně vyřešen tak, že v případě delšího vstupu je vstup rozdělen na několik částí a jednotlivé části prochází modelem sekvenčně. Tento přístup funguje v pořádku s většinou vstupů, ovšem jsou zde i případy, kdy nepřítomnost celého kontextu strany vede ke špatné klasifikaci. Příklady takového zpracování lze vidět na obrázcích 11 a 12.

360		
VII.	Pobělohorské stíny	177
	„Zimní“ král český	179
	Růžový palouček	185
	Do vyhnanství!	188
	Jezovitské prostředky	192
	Před příchodem „sv. misie“	196
	Jezovitské kázání	201
	Koniáš pálí „kacířské“ knihy české	208
	Pronásledování „bratří“	212
	Jezovitě triumfuji	215
VIII.	Z dob utrpení česko-slovenského lidu	217
	Přástky na Chodsku	219

Obrázek 11: První zpracovaná dávka strany. Všechna slova (kromě některých teček) jsou zde označena korektně, správně jsou rozlišeny kapitoly od podkapitol.

	V Praze před sto lety	288
	Apoštol mateřského jazyka	289
	O hrozných nestvůrách	295
	Věk hájí své mateřštinu	300
	O české divadlo	305
	U Josefa Dobrovského	309
	Signum a jiné trápení	313
X.	Na prahu nové doby	319
	V Litomyšli před sto lety	321
	Jak učila M. D. Rettigová vlastenectví	324
	Majales v Litomyšli	330
	Červánky svobody	340

Obrázek 12: Čtvrtá zpracovaná dávka strany. U této dávky model již ztratil kontext o předcházejících kapitolách, proto nesprávně klasifikuje podkapitoly jako kapitoly a zkaližuje se až po další opravdové kapitole.

7.2.5 Převod výstupů do formátu JSON

Pro potřeby inference máme k dispozici data z OCR: slova, jejich bounding boxy a klasifikace jednotlivých slov. Následující pipeline představuje výsledek řady experimentů a kompromisů s cílem zpracovat tato data co nejkorektněji pro širokou škálu vstupů/dokumentů.

Během implementace jsme zvažovali různé přístupy, například průchod po řádcích podle formátu ALTO (výstup PERO-OCR). Tímto postupem lze v ideálním případě získat slova uspořádaná podle logických řádků, což je zpravidla požadovaný výstup. Ukázkou ideálního vstupu pro tento přístup ilustruje obrázek 13. Tento přístup však nelze spolehlivě použít u různorodých dokumentů, protože výstup OCR (ve formátu ALTO) text často dělí na tzv. *TextBlocks*, které v mnoha případech odpovídají spíše sloupovému rozvržení (např. nadpis, číslo strany a kapitoly umístěné vedle sebe), nikoli logickým řádkům, jak ukazuje obrázek 14. Jednoduchý průchod po "řádci" definovaných těmito bloky je pak problematický.

Dalším problémem byla spolehlivá detekce jedno- či víceřádkových nadpisů a tomu odpovídající parsování. Zkoušeli jsme různé přístupy, například zjišťování, zda další řádek začíná odsazeně vzhledem k předchozímu, ale to nefungovalo příliš dobře. Nespolehlivým se ukázalo i získávání této informace podle toho, zda daný řetězec má v blízkosti číslo kapitoly, především proto, že ne všechny nadpisy mají v těsné blízkosti čísla kapitol.

OBSAH.	
	Str.
Starosvětské obrázky	9
Ztracenci	31
Z paměti samotářových	49
Z literární historie	187
Ze Slovenska	217
Za »Bratrstvem«	239
Černá legie v Čechách	253
Zikmund Winter	265

Obrázek 13: Ukázka vstupu vhodného pro zpracování po řádcích podle výstupu OCR.

	Seite
40. Die Ausrichtung zur Schule, von Mathella Braun	47
41. Der Vater und die drei Söhne, von Richter	51
42. Zusammengelebte Sätze. (Subjekt- und Prädikatsätze)	53
43. Fabeln, von Feyta	54
44. Das Pferd und der Fiel, von Klein	55
45. Mehr den Baum hin! Von Schmid	56
46. Mahnung. Nach Walther v. der Vogelweide von Haag	57
47. Anekdoten	58
48. Zusammengelebte Sätze. (Adverbialsätze)	59

Obrázek 14: Ukázka nevhodně rozdělených TextBlock bloků (znázorněno modře) znemožňující průchod po logických řádcích podle výstupu OCR.

Značné potíže působilo také určení, zda k danému nadpisu náleží číslo stránky a kapitoly, a pokud ano, ke kterému konkrétnímu nadpisu. Pokusy přiřadit čísla k nejbližšímu nadpisu selhaly. To se ukázalo jako nespolehlivé především kvůli proměnlivé délce nadpisů (jednorádkové vs. víceřádkové), sloupovému rozvržení a dalším faktorům, které ovlivňovaly výpočet "nejbližšího".

Po důkladné analýze výstupů a řadě experimentů se zpracováním jsme dospěli k závěru, že pro široce různorodé dokumenty není možné v rámci tohoto projektu navrhnout čistě pravidlový systém, který by fungoval spolehlivě bez významných kompromisů. Řešením by bylo buď jasné vymezení vstupní domény (např. co do počtu sloupců, struktury nadpisů atd.), nebo využití technik strojového učení pro extrakci dat do JSONu. To by mohla být klasifikace typů obsahu (nadpis, odstavec, číslo) nebo kompletní end-to-end model. Tyto přístupy však nebyly předmětem/cílem tohoto projektu, a proto nebyly implementovány.

Výsledný (implementovaný) přístup se proto snaží minimalizovat kompromisy a dosáhnout rozumné funkčnosti pro co nejširší spektrum vstupů. Skládá se z následujících kroků:

1. Filtrace relevantních slov (např. nadpisů, čísel stran, čísel kapitol) na základě jejich klasifikace.
2. Spojování jednotlivých slov do vět (víceslovné nadpisy, vícetřířná čísla atd.).
 - (a) Vybraná slova se zpracovávají podle jejich klasifikace.
 - (b) Pro slova v rámci stejné třídy se provede pokus o spojení. Slova se spojí, pokud leží na přibližně stejné horizontální úrovni a jejich horizontální vzdálenost (včetně mezer) nepřesahuje určitou mez. Tato mez je relativní vzhledem k výšce bounding boxu slova (de facto výška řádku).
 - (c) Postup se opakuje, dokud nejsou zpracována všechna slova dané třídy/všechn relevantní text.
3. Vytvoření relací mezi nadpisy a příslušnými čísly stran či kapitol (pokud existují).
 - (a) Pro každý nadpis se určí jeho horizontální středová linie.
 - (b) Pro každé slovo klasifikované jako číslo strany nebo číslo kapitoly se zjistí, zda jeho vertikální rozsah (bounding box) protíná horizontální středovou linii některého nadpisu.
 - (c) Mezi protínajícími se kandidáty (číslly) a daným nadpisem se spočítá euklidovská vzdálenost jejich středů (zvlášť pro čísla stran a zvlášť pro čísla kapitol).
 - (d) Jako příslušné číslo (strany/kapitoly) je vybrán ten kandidát s nejmenší euklidovskou vzdáleností od nadpisu.
4. Určení nadřazeného (hlavního) nadpisu pro dané (pod)nadpisy na základě horizontální blízkosti či odsazení.
5. Sestavení výsledného JSON objektu z extrahovaných a propojených informací.

8 Vyhodnocení přístupu s LayoutLMv3

Pro vyhodnocení jsme si napsali script dostupný v odevzdaném repozitáři. Tento script porovnal GT JSON souborů¹⁰ s hodnotami predikovanými pipeline modelu LayoutLMv3. Pro porovnání s DONUT modelem jsme použili stejný dataset jako pro validaci DONUTu a taktéž nás zajímalo průměrné TED skóre a F1 skóre. Obě tyto hodnoty jsou počítány maximálně podobným způsob jak v modelu donut. Bohužel úplně přesně stejného výpočtu nejde dosáhnout, protože v DONUT modelu jsou hodnoty metrik počítané před konverzí strukturovaného výstupu na JSON.

Počet epoch LayoutLMv3	Norm. tree edit distance přesnost	F1 skóre
10	0.73	0.19

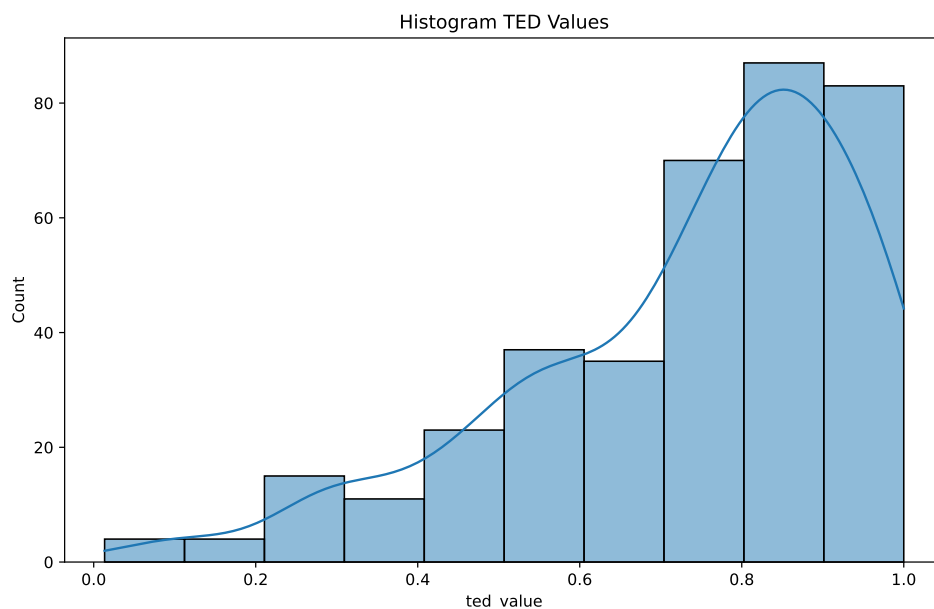
Tabulka 6: Tabulka s výsledky inference pipeline založené na LayoutLMv3.

Pro lepší pochopení dat jsme si vykreslili histogram rozložení na obrázku 15. Z rozložení dat je zjevné, že navržený systém funguje relativně dobře. Vysoká přesnost TED indikuje, že se systému relativně dobře daří předpovídat strukturu, což ale není příliš překvapivé, protože dataset obsahuje relativně jednoduché obsahy. Dále pak proto, že jsou málo časté kombinace vícesloupcových či víceřádkových nadpisů, u kterých víme, že je systém náchylný k chybám, které v datasetu téměř

¹⁰Získané LLM a validované ručně

nejdou. Mimo to má systém předpovídání struktury značně jednodušší, protože struktura se nemusí hádat, ale pouze "nasazuje" na získané informace.

Nízká přesnost F1 naproti tomu jasně ukazuje na nedokonalý postprocessing, který, jak jsme nastínili dříve, by šlo vylepšit pouze za cenu obrovské complexity nebo použitím principiálně jiného modelu.



Obrázek 15: Histogram rozložení TED hodnot v datasetu.

9 Závěr

Cílem tohoto projektu bylo prozkoumat a implementovat systémy pro automatické zpracování skenovaných stránek s obsahem knih a dokumentů a extrakci strukturovaných informací o kapitolách, jejich číslech, číslech stránek a hierarchickém vztahu. Pro dosažení tohoto cíle jsme se zaměřili na dva hlavní přístupy: end-to-end model založený na architektuře Transformer (DONUT) a sekvencí řešení kombinující optické rozpoznávání znaků (PERO OCR) s modelem pro porozumění dokumentům založeným na rozložení naskenované strany (LayoutLMv3).

Pro model DONUT jsme se rozhodli pro přístup k tvorbě anotací pomocí velkého jazykového modelu (LLM). V druhé části semestru jsme zvalidovali přesnost dat pro validaci přesnosti. Data pro PERO OCR + LayoutLMv3 přístup vyžadovala jiný typ anotací, zaměřený na klasifikaci slovních tokenů na základě jejich pozice a bounding boxů, což představovalo odlišné výzvy.

Oba přístupy jsme vyhodnotili pomocí standardních metrik – normalizované Tree Edit Distance a F1 skóre. I přes počáteční potíže s adaptací modelu DONUT, který nebyl trénován na českém jazyce, a nutnost úprav jeho kódu pro dosažení stabilních výsledků trénování, model DONUT v našem testování dosáhl pro danou úlohu lepších výsledků ve srovnání s řešením PERO OCR + LayoutLMv3.

Závěrem lze říci, že ačkoliv přístup založený na PERO OCR a LayoutLMv3 poskytuje detailní klasifikaci na úrovni slov a mohl by být užitečný pro jiné úlohy zpracování dokumentů, pro extrakci strukturovaného obsahu z obrazových dat se v rámci tohoto projektu ukázal jako efektivnější model DONUT. Primární problém PERO + LayoutLMv3 řešení totiž leží v posledním kroku zpracování, kdy je výstup modelu přetvářen na výstupní JSON, který nelze s dostatečnou přesností rozumně realizovat pomocí standardního programování. Proto pro další možné směry, kterými by se toho řešení mohlo ubírat, by mohlo být např. vyzkoušení natrénování neuronové sítě pro sestavení výstupů.

End-to-end povaha našeho baseline modelu DONUT a možnost rychlé tvorby dat s asistencí LLM, byť s nutností ruční kontroly, představovala jednodušší cestu pro řešení úloh porozumění dokumentům přímo z jejich obrazové reprezentace. Navíc, DONUT bylo možné trénovat na větším datasetu z důvodu nepotřeby anotací, a díky tomu jsme jej také mohli trénovat na stránkách s více než dvojúrovňovým zanořením obsahu.

Reference

- [1] Geewook Kim¹, Teakgyu Hong, Moonbin Yim, Jeongyeon Nam¹, Jinyoung Park et al. Ocr-free document understanding transformer. <https://arxiv.org/abs/2111.15664>, 2022.
- [2] Oldřich Kodým, Michal Hradíš. Page layout analysis system for unconstrained historic documents. <https://arxiv.org/abs/2102.11838>, 2021.
- [3] Yupan Huang, Tengchao Lv, Lei Cui, Yutong Lu, Furu Wei. Layoutlmv3: Pre-training for document ai with unified text and image masking. <https://arxiv.org/abs/2204.08387>, 2022.